

Pandas — Data Cleaning

13 topics with simple examples, line-by-line code, and expected output.

1. Missing Values

Use `isna()` to find missing values and `fillna()` to replace them.

Code — line by line

```
import pandas as pd

df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [20, None, 30]
})

print(df.isna())

df["Age"] = df["Age"].fillna(df["Age"].mean())
print(df)
```

Output

	Name	Age
0	False	False
1	False	True
2	False	False

	Name	Age
0	A	20.0
1	B	25.0
2	C	30.0

2. Duplicate Values

`uplicated()` identifies repeated rows; `drop_duplicates()` removes them.

Code — line by line

```
df = pd.DataFrame({
    "Name": ["A", "B", "B", "C"],
    "Age": [20, 25, 25, 30]
})

print(df.duplicated())
print(df.drop_duplicates())
```

Output

0	False
1	False
2	True
3	False

dtype: bool

	Name	Age
0	A	20
1	B	25
3	C	30

3. Wrong Data Types

Convert numbers stored as text into numeric data with `pd.to_numeric()`.

Code — line by line

```
df = pd.DataFrame({
    "Age": ["20", "25", "30"]
})

print(df.dtypes)

df["Age"] = pd.to_numeric(df["Age"])
print(df.dtypes)
```

Output

Age	object
-----	--------

```
dtype: object
```

```
Age      int64  
dtype: object
```

4. Incorrect Values

Check a business rule and replace invalid values. Here, negative age is invalid.

Code — line by line

```
import numpy as np  
  
df = pd.DataFrame({  
    "Name": ["A", "B", "C"],  
    "Age": [20, -5, 30]  
})  
  
df.loc[df["Age"] < 0, "Age"] = np.nan  
print(df)
```

Output

	Name	Age
0	A	20.0
1	B	NaN
2	C	30.0

5. Outliers

The IQR method finds unusually high or low values.

Code — line by line

```
df = pd.DataFrame({  
    "Salary": [30000, 32000, 35000, 34000, 36000, 200000]  
})  
  
Q1 = df["Salary"].quantile(.25)  
Q3 = df["Salary"].quantile(.75)  
IQR = Q3 - Q1  
  
lower = Q1 - 1.5 * IQR  
upper = Q3 + 1.5 * IQR  
  
print("Q1:", Q1)  
print("Q3:", Q3)  
print("IQR:", IQR)  
print("Lower:", lower)  
print("Upper:", upper)  
  
print(df[(df["Salary"] < lower) | (df["Salary"] > upper)])
```

Output

```
Q1: 32500.0  
Q3: 35750.0  
IQR: 3250.0  
Lower: 27625.0  
Upper: 40625.0  
Salary  
5    200000
```

6. Text Cleaning

Remove extra spaces and standardize text case.

Code — line by line

```
df = pd.DataFrame({  
    "City": [" Pune ", "MUMBAI ", " Delhi"]  
})  
  
df["City"] = df["City"].str.strip().str.lower()  
print(df)
```

Output

	City
--	------

```
0    pune
1    mumbai
2    delhi
```

7. Numeric Cleaning

Remove currency symbols and commas, then convert text to integers.

Code — line by line

```
df = pd.DataFrame({
    "Sales": ["₹1,200", "₹2,500", "₹3,000"]
})

df["Sales"] = (df["Sales"]
    .str.replace("₹", "", regex=False)
    .str.replace(",", "", regex=False)
    .astype(int))

print(df)
print(df["Sales"].dtype)
```

Output

```
      Sales
0    1200
1    2500
2    3000
int64
```

8. Date Cleaning

Convert date text into Pandas datetime values.

Code — line by line

```
df = pd.DataFrame({
    "Date": ["2026-01-10", "2026-02-15", "2026-03-20"]
})

df["Date"] = pd.to_datetime(df["Date"])
print(df)
print(df["Date"].dtype)
```

Output

```
      Date
0 2026-01-10
1 2026-02-15
2 2026-03-20
datetime64[ns]
```

9. Standardizing Values

Convert different spellings or abbreviations into one consistent value.

Code — line by line

```
df = pd.DataFrame({
    "Gender": ["Male", "M", "male", "Female", "F"]
})

df["Gender"] = df["Gender"].replace({
    "M": "Male",
    "male": "Male",
    "F": "Female"
})

print(df)
```

Output

```
      Gender
0      Male
1      Male
2      Male
3    Female
4    Female
```

10. Renaming Columns

Use `rename()` to make column names clear.

Code — line by line

```
df = pd.DataFrame({
    "nm": ["A", "B"],
    "ag": [20, 25]
})

df = df.rename(columns={
    "nm": "Name",
    "ag": "Age"
})

print(df)
```

Output

	Name	Age
0	A	20
1	B	25

11. Removing Unwanted Columns

Use `drop(columns=...)` to remove unnecessary columns.

Code — line by line

```
df = pd.DataFrame({
    "Name": ["A", "B"],
    "Age": [20, 25],
    "Temp": [100, 200]
})

df = df.drop(columns=["Temp"])
print(df)
```

Output

	Name	Age
0	A	20
1	B	25

12. Removing Unwanted Rows

Use boolean filtering to keep only rows that satisfy a condition.

Code — line by line

```
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [20, 15, 25]
})

df = df[df["Age"] >= 18]
print(df)
```

Output

	Name	Age
0	A	20
2	C	25

13. Data Validation

Validate data against an expected rule. Here, valid age must be 18–60.

Code — line by line

```
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Age": [20, 70, 25]
})

valid = df["Age"].between(18, 60)

print(valid)
```

```
print("\nInvalid rows:")
print(df[~valid])
```

Output

```
0      True
1     False
2      True
Name: Age, dtype: bool
```

```
Invalid rows:
   Name  Age
1     B   70
```

Quick Revision

Missing → `isna()`, `fillna()` | Duplicates → `duplicated()`, `drop_duplicates()` | Types → `astype()`, `to_numeric()`, `to_datetime()` | Incorrect values → `conditions + loc[]` | Outliers → `quantile()` + IQR | Text → `str.strip()`, `str.lower()`, `str.replace()` | Numeric → `replace symbols + astype()` | Dates → `to_datetime()` | Standardize → `replace()`, `map()` | Columns → `rename()`, `drop()` | Rows → `filtering` | Validation → `between()` and `conditions`.